applied sciences

MDPI

Article

An Efficient Blockchain-Based Data Storage and Query System for Large-Scale Data Sharing

Jiangbing Yang, Hui Li * and Ming Zhang *

School of Cyber Engineering, Xidian University, Xi'an 710071, China; zzyjb@stu.xidian.edu.cn
* Correspondence: lihui@mail.xidian.edu.cn (H.L.); zhangming2025@njust.edu.cn (M.Z.)

Abstract

In blockchain–IPFS-based systems, full nodes maintain complete ledger replicas, whereas light nodes retain only essential information such as block headers to reduce storage and computation overhead. Due to the absence of full data replicas, light nodes are unable to support full-data queries, which limits their applicability in practical financial data sharing scenarios. Moreover, conventional blockchain storage mechanisms rely on synchronous confirmation across multiple nodes, resulting in limited throughput and poor responsiveness under high-concurrency and burst-traffic conditions. To address these issues, this paper proposes a blockchain–IPFS-based storage and query scheme for banking credit data that integrates multi-level caching, non-blocking asynchronous processing, and a Cuckoo filter–based lightweight query mechanism. The proposed scheme enables light nodes to efficiently verify the existence of credit files and retrieve associated metadata without maintaining complete ledger replicas, while a coordinated caching–asynchronous architecture decouples user requests from on-chain and off-chain persistence operations to improve system throughput and robustness. A prototype system is implemented and evaluated under varying file sizes and concurrency levels. Experimental results show that, for files smaller than 100 MB, the proposed scheme reduces storage latency by approximately 35–99% and improves query response time by more than 95%, compared with conventional blockchain–IPFS-based solutions. In addition, download latency is reduced by 20–31% for small and medium-sized files. The results further confirm that the proposed approach effectively supports full-data queries for light nodes and demonstrates strong resilience under burst traffic conditions, indicating its practical feasibility for secure financial credit data sharing.

Keywords: blockchain; data sharing; blockchain storage; storage performance

Check for updates

Academic Editor: Gianluca Lax

Received: 19 January 2026
Revised: 20 February 2026
Accepted: 6 March 2026
Published: 12 March 2026

Copyright: © 2026 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license.

1. Introduction

With the continuous acceleration of global digitalization [1,2], data have become a core driver of financial innovation. Among various data assets, banking credit data—serving as fundamental information for assessing customer creditworthiness, regulating financial behavior, and supporting banks' risk management and credit decision-making—play an indispensable role in enhancing risk control capabilities and maintaining the security and stability of financial markets.

However, constrained by heterogeneous system architectures, disparate security mechanisms, and organizational collaboration barriers, banking credit data remain largely isolated in so-called data silos, making efficient, trustworthy, and secure data sharing difficult to achieve [3,4]. This fragmentation not only restricts the development of the

banking industry itself but also poses adverse implications for the broader economic and social system.

For instance, during asset certification for individual users, financial assets are often distributed across multiple banking institutions. Owing to the lack of direct interoperability among banking information systems, users are required to complete certification procedures separately at each institution. From the user perspective, this process is time-consuming and labor-intensive, significantly degrading the service experience. From the bank perspective, incomplete access to comprehensive customer credit information may lead to biased risk assessments and suboptimal credit decisions. In cross-bank lending scenarios, information asymmetry may further cause individual banks to overextend credit, thereby increasing credit risk. At a systemic level, delayed or incomplete data sharing can contribute to excessive lending across institutions, potentially amplifying systemic financial risks when borrowers are unable to repay accumulated debts at maturity.

To provide a clearer context for this study, a focused literature review was conducted. Representative studies on blockchain–IPFS-based data storage and sharing were selected from peer-reviewed academic publications. This review does not aim to be exhaustive; instead, it emphasizes typical architectures, design approaches, and technical limitations that are most relevant to practical banking credit data sharing scenarios. The strengths and weaknesses of these representative solutions serve as the primary basis for the design of the proposed system. To mitigate these challenges, both academia and industry have increasingly explored the application of blockchain technology in banking credit data sharing.

Existing blockchain–IPFS-based credit data storage and sharing studies can be broadly categorized into three representative research directions.

The first line of work focuses on privacy-preserving credit data sharing by integrating blockchain with advanced cryptographic or privacy-computation techniques. Dong Chunbo et al. [5] combined federated learning with blockchain to enable decentralized credit data collaboration under de-identification and anonymization constraints. Similarly, Lu Shengyu et al. [6] proposed a blockchain–IPFS-based credit data management framework incorporating privacy computation to support secure credit data utilization. These approaches effectively enhance data privacy and regulatory compliance; however, they typically assume that participating nodes can access sufficient on-chain or off-chain data replicas and do not explicitly address the query limitations of blockchain light nodes that maintain only partial ledger information.

The second research direction leverages blockchain to improve trust, traceability, and auditability in credit data sharing platforms. Ju et al. [7] integrated blockchain with big data technologies to design a decentralized lending application, aiming to reduce information asymmetry in credit platforms. More broadly, blockchain's inherent properties—such as tamper resistance, traceability, and verifiability—have been widely recognized as essential enablers for trustworthy credit data sharing [8–10]. While these studies significantly strengthen data integrity and accountability, most of them rely on synchronous blockchain storage and consensus mechanisms, which suffer from limited throughput and high latency under high-concurrency or burst-traffic conditions.

The third category explores the combination of blockchain with distributed storage systems, such as IPFS, to alleviate on-chain storage pressure. By storing large-scale credit data off-chain while maintaining only metadata or content hashes on the blockchain, blockchain–IPFS-based solutions improve storage scalability and reduce on-chain overhead. Nevertheless, existing frameworks often overlook two practical issues: (1) blockchain light nodes are unable to support full-data queries due to incomplete ledger replicas, and

(2) the absence of effective caching and asynchronous processing mechanisms leads to performance degradation and system instability under concentrated access workloads.

In summary, although existing studies make important progress in privacy protection, data integrity, and scalable storage, they still fall short in simultaneously supporting efficient full-data queries for light nodes and maintaining high system performance under bursty access patterns. To address these limitations, this work proposes a blockchain–IPFS-based storage and query architecture that integrates a Cuckoo filter–based lightweight query mechanism, multi-level caching, and asynchronous processing, thereby enhancing query capability, throughput, and robustness in practical banking credit data sharing scenarios.

The main contributions of this study are summarized as follows.

First, this work addresses a practical yet often overlooked limitation of blockchain–IPFS-based storage systems: blockchain light nodes are unable to perform comprehensive data queries due to the absence of full ledger replicas. To overcome this issue, a lightweight full-data query scheme based on a Cuckoo filter is proposed. By preserving compact file fingerprints and essential metadata, the proposed scheme enables light nodes to efficiently verify file existence and retrieve associated information without maintaining complete blockchain data.

Second, a multi-level storage architecture combining distributed caching and asynchronous processing is designed to improve system performance under high concurrency. By decoupling user requests from on-chain and off-chain persistence operations, the proposed architecture significantly reduces storage latency and enhances query and download efficiency, while maintaining data integrity and consistency.

Third, extensive experimental evaluations are conducted to analyze system performance across different file sizes and concurrency levels. The results demonstrate that the proposed scheme achieves substantial performance improvements for small and medium-sized files and exhibits strong resilience under burst traffic conditions. Moreover, the experiments reveal an important engineering insight: although caching mechanisms effectively improve performance in most scenarios, they may introduce storage overhead for large files, highlighting a critical trade-off that has not been sufficiently discussed in existing blockchain–IPFS-based storage studies.

The remainder of this paper is organized as follows. Section 2 reviews the related work relevant to this study, including blockchain technology, the InterPlanetary File System (IPFS), Cuckoo filters, and existing research on data sharing. Section 3 presents the overall system architecture and details the storage workflow of user credit data. Section 4 describes the key system designs, including the multi-level caching mechanism and the lightweight query scheme for blockchain light nodes, and introduces the design of the Cuckoo filter. Section 5 reports the experimental setup and evaluation results under different file sizes and workload conditions, and further analyzes the query complexity and the system overhead introduced by the asynchronous processing mechanism. Finally, Section 6 concludes the paper and discusses potential directions for future work.

2. Related Work

2.1. Blockchain Technology

Blockchain technology originated from the Bitcoin white paper published by Satoshi Nakamoto in 2008 [11]. As a highly innovative form of distributed ledger technology, blockchains can be broadly classified—according to their application scope—into three categories: public blockchains, consortium (permissioned) blockchains, and private blockchains [12].

Public blockchains, with Bitcoin as a representative example, are typically characterized by a high degree of openness. Any participant can create an account, initiate

transactions, and participate in consensus. All transactions are transparent and publicly verifiable. However, because public blockchains must process a large volume of open data and transactions, they generally suffer from limited throughput and relatively weak privacy protection.

In contrast, private blockchains provide strong privacy guarantees by restricting participation to specific users or organizations. Benefiting from fully customizable consensus mechanisms, private blockchains usually achieve higher operational efficiency than public blockchains. Nevertheless, their closed and centralized governance often results in higher maintenance costs and limited innovation potential.

Consortium blockchains lie between public and private blockchains, aiming to strike a balance between openness and privacy. They are jointly maintained and governed by multiple organizations or institutions, making them particularly suitable for inter-organizational collaboration scenarios. Hyperledger Fabric [13,14], initiated by IBM, is a representative consortium blockchain framework. It provides a modular, pluggable, and scalable architecture for distributed ledger solutions, allowing users to customize and deploy smart contracts according to application-specific requirements. Moreover, Hyperledger Fabric supports the creation of channels, which enable participants to establish logically isolated ledgers. Only members within the same channel are authorized to access the corresponding ledger data, thereby offering enhanced privacy protection. Owing to these features, Hyperledger Fabric has been widely adopted as an effective solution for secure, privacy-preserving data sharing and collaboration in enterprise environments.

2.2. InterPlanetary File System (IPFS)

The InterPlanetary File System (IPFS) is a peer-to-peer distributed file system developed to address the limitations of conventional centralized storage architectures. In IPFS, a file is divided into multiple smaller blocks, and each block is assigned a unique cryptographic hash as its identifier. These content-based identifiers not only support data deduplication but also enable precise retrieval of file fragments across the network.

When a file is requested, its hash value is used to locate the corresponding blocks stored on different nodes. The blocks are then fetched through peer-to-peer communication and reassembled to reconstruct the original file. By leveraging content-addressable storage together with decentralized networking, IPFS forms a distributed storage framework in which data are maintained across multiple nodes rather than relying on a single centralized server.

This decentralized architecture provides IPFS with excellent scalability in terms of storage capacity, as additional nodes can be dynamically integrated to contribute storage resources. Consequently, IPFS is particularly well suited as an off-chain storage complement for blockchain systems, effectively mitigating on-chain storage constraints while preserving data integrity, availability, and accessibility.

Beyond its use as a storage substrate, recent studies have also analyzed IPFS's architectural trade-offs. For example, Trautwein et al. [15] provided an in-depth evaluation of IPFS performance, highlighting trade-offs between retrieval latency and network overhead. Such analyses demonstrate that while IPFS enhances scalability, careful consideration of caching strategies remains essential for high-performance applications.

2.3. Cuckoo Filter and Probabilistic Data Structures

Probabilistic data structures are widely used in distributed systems for fast membership queries with low storage overhead. Among these probabilistic data structures, the Bloom filter [16] is a popular method. It has the benefit of compact storage and efficient lookups. However, the conventional Bloom filter is not meant to support delete operations.

Although some variants have been proposed to address this limitation, they sometimes introduce penalties in terms of additional storage or higher complexity.

The Cuckoo filter [17–19] is a compact probabilistic data structure that can be employed to assess the likelihood of an element belonging to a set. It is based on Cuckoo hashing and retains its displacement-based insertion approach. The name "Cuckoo filter" comes from the habit of cuckoo birds of taking over other birds' nests, which is a metaphor for the displacement process used to resolve collisions.

To understand the Cuckoo filter, it is helpful to briefly revisit Cuckoo hashing. In its basic form, a cuckoo hash table consists of an array structure associated with two independent hash functions. Each element is mapped to two candidate locations. If either position is empty, the element can be placed directly. When both positions are occupied, an existing element is randomly selected and evicted (i.e., "kicked out") to make room for the new element. The displaced element is then reinserted by attempting to occupy its alternative hash position. This eviction–reinsertion process may repeat until an empty slot is found. If the table becomes overly congested and excessive evictions significantly degrade performance, the hash table is resized, and all elements are rehashed and reinserted.

Building on this mechanism, the Cuckoo filter adopts a compact array structure in which each bucket stores multiple fingerprints instead of complete elements. In a typical configuration, a bucket occupies 4 bytes and can hold up to four fingerprints. This layout reduces the probability of cyclic evictions and improves insertion stability. Since only fingerprint information is stored, the Cuckoo filter achieves significantly better space efficiency than conventional Cuckoo hashing while maintaining an acceptable false positive rate.

Compared with full-data indexing or direct storage approaches, the Cuckoo filter introduces limited computational overhead and supports efficient membership testing through simple hash operations. It also naturally avoids redundant insertions. More importantly, unlike standard Bloom filters, it allows dynamic deletion without requiring reconstruction of the entire structure. When combined with a distributed caching layer, these characteristics enable efficient file-existence verification and reduce unnecessary access to backend storage systems. As a result, the Cuckoo filter is well suited for dynamic indexing scenarios in distributed environments where storage efficiency and update flexibility are both required.

2.4. Existing Decentralized Data Sharing Schemes

Recently, the topic of decentralized data storage has gained more and more attention. According to the comprehensive surveys [20–22], the "Blockchain + IPFS" framework has become a widely accepted standard for data-intensive applications. However, according to the surveys, most existing solutions are based on synchronous interactions between the on-chain and off-chain parts, which leads to performance bottlenecks.

In view of the challenges faced by blockchain–IPFS-based file sharing, scholars have proposed specific implementation plans. Chen Lianggui et al. [23] integrated on-chain and off-chain storage with priority-based ranking. Fan Xianli et al. [24] focused on privacy-preserving supply chains. Rui Kunrui et al. [25] proposed recording only file hashes on-chain to alleviate storage pressure. Jun Wook Heo et al. [26] introduced a multi-level distributed caching strategy based on access frequency. Li Chunlin et al. [27] optimized internal blockchain consensus structures. Ruinian Li et al. [28] focused on certificate-less cryptography for IoT. Dai Mingjun et al. [29] utilized network coding to reduce storage space.

Despite the great progress and achievements mentioned in the existing studies, there are still some limitations in the existing decentralized data storage systems. Although many

existing studies have proposed solutions that combine on-chain and off-chain storage to alleviate the blockchain storage overhead and partly solve the scalability problem, few studies have considered efficient full-data querying for lightweight nodes. In addition, most existing [23,25] solutions are based on direct access to IPFS in a synchronous way, which is inefficient for bursty access patterns and sudden traffic spikes. To overcome the above limitations, this paper proposes a storage and query solution that combines a Cuckoo filter indexing scheme with multi-level caching and asynchronous processing.

3. Blockchain and IPFS-Based Data Storage System

3.1. System Architecture Overview

The storage architecture of the proposed system is composed of two complementary components: on-chain storage and off-chain storage. The on-chain layer is mainly responsible for maintaining descriptive metadata associated with credit files, including file indices, hash values, file names, keywords, annotations, and other relevant attributes. In contrast, the off-chain layer is dedicated to storing the encrypted credit files themselves.

By integrating blockchain technology, the InterPlanetary File System (IPFS), and a caching mechanism, the proposed architecture jointly achieves secure data storage and efficient data sharing. Specifically, blockchain technology ensures data immutability and traceability, IPFS provides decentralized and scalable file storage, and the caching mechanism significantly improves system performance by reducing redundant access to the blockchain and IPFS networks.

Through the coordinated design of on-chain and off-chain storage, the system simultaneously satisfies security, efficiency, and scalability requirements. This hybrid architecture enables reliable data preservation while supporting fast and flexible data access, thereby meeting the needs of diverse financial application scenarios. The overall system architecture is illustrated in Figure 1.

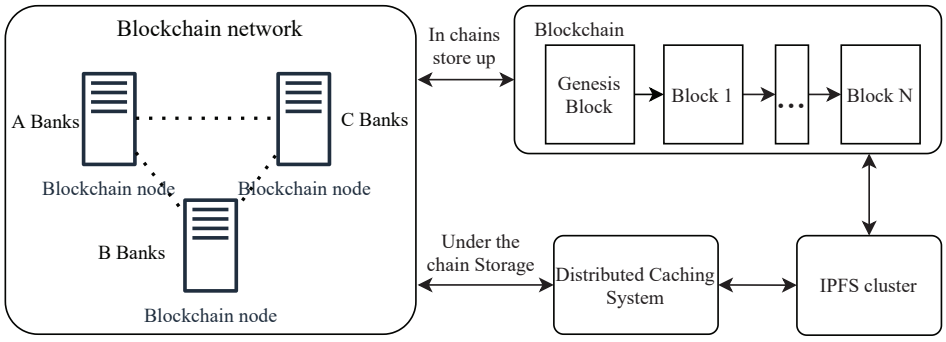


Figure 1. Overall system design.

The system architecture is organized into four layers: the API layer, the capability layer, the caching layer, and the data layer. Each layer is assigned well-defined responsibilities, and the layers collaborate closely to ensure stable and reliable system operation. The overall system architecture is illustrated in Figure 2.

The API layer functions as the unified entry point of the system. It encapsulates internal functional modules and, based on the services provided by the capability layer, integrates these functionalities to expose standardized and unified service interfaces to external applications. The capability layer represents a fine-grained decomposition of the system's core service functions. Compared with the API layer, this layer provides services with narrower scope and finer granularity. This design enhances the reusability and composability of system functions, thereby facilitating the construction of modules with high cohesion and low coupling.

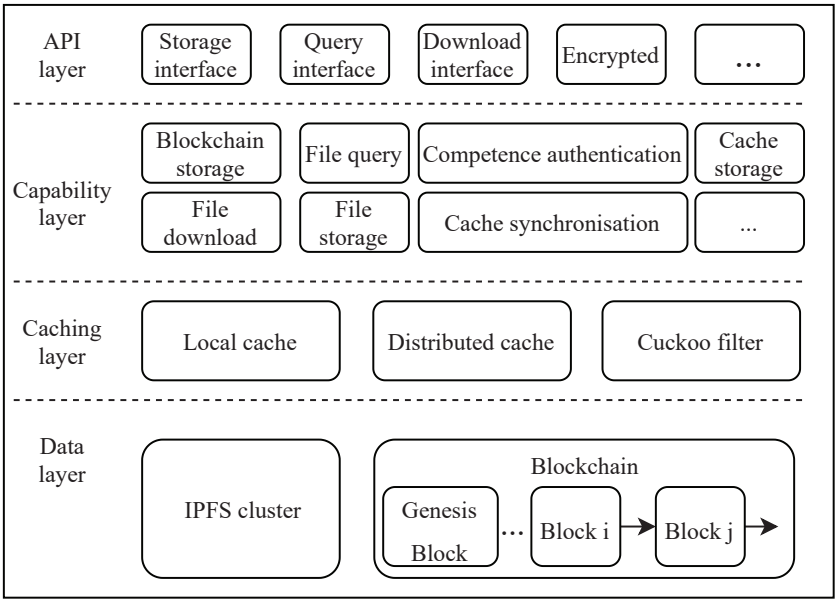


Figure 2. System Architecture Diagram.

The caching layer is mainly responsible for storing frequently accessed data to improve data retrieval efficiency. By leveraging high-performance storage media (e.g., in-memory caching) and positioning cached data closer to either the data source or the request endpoints, this layer significantly improves overall system performance and availability.

The data layer is further divided into two major components. The first component is the IPFS cluster, which is responsible for storing the actual encrypted file contents. The second component is the blockchain system, whose primary role is to maintain critical metadata, including file indices, hash values, and file names. By ensuring the immutability and tamper resistance of this metadata, the blockchain system guarantees the security and integrity of the stored data.

To better illustrate the logic of the execution process of the proposed system, Algorithm 1 gives a summary of the overall workflow of the credit data storage and query process, including cache interaction, asynchronous processing, blockchain recording, IPFS storage, and lightweight query operations using the Cuckoo filter.

After introducing the overall system architecture, it is also necessary to further illustrate the key novelty of the proposed system design compared to the existing blockchain–IPFS storage system architectures. In most existing architectures, the blockchain is mainly used to store the file hash or metadata, and file access is accomplished by direct and synchronous access to the IPFS. In these architectures, it is generally assumed that full nodes are involved or that on-chain queries are made frequently, which greatly hinders the query functionality of light nodes and causes performance degradation in scenarios with high concurrency or burst access.

In contrast, the proposed system presents a system-level design that integrates multi-level caching, non-blocking asynchronous processing, and a Cuckoo filter-based lightweight indexing solution. Compared with traditional cache designs that only store metadata, the proposed system's caching layer stores the encrypted credit files directly, which enables fast data access without requiring immediate access to the blockchain or IPFS. Meanwhile, the asynchronous task queues are used to decouple user requests from on-chain recording and off-chain storage, which reduces response latency and improves the overall throughput of the system. The Cuckoo filter also enables light nodes to efficiently verify file existence with low storage overhead, without requiring unnecessary blockchain and IPFS queries.

These components are coordinated together by a unified storage and query workflow, which is summarized in Algorithm 1. By coordinating these components together, the proposed system is able to achieve the simultaneous functionality of supporting lightweight full-data queries, improving performance in high concurrency scenarios, and enhancing robustness in burst traffic access scenarios, which makes the proposed system different from existing blockchain–IPFS storage system architectures.

Algorithm 1: Overall Workflow of the Proposed Blockchain–IPFS-Based Storage and Query Framework

Input : Credit data file F, user request R
Output: Transaction hash $TxID$, query response Q_r
Initialization: Initialize multi-level cache $\mathcal{C}$, asynchronous task queue $\mathcal{Q}$, and Cuckoo filter $\mathcal{CF}$;
if *R is a data upload request* **then**
  Generate file hash $h = \text{Hash}(F)$;
  Extract metadata M (owner, timestamp, permissions);
  Store (h, F, M) into cache $\mathcal{C}$;
  Insert fingerprint of h into $\mathcal{CF}$;
  Push asynchronous storage task (F, h, M) into queue $\mathcal{Q}$;
  Asynchronously execute:;
    Upload F to IPFS and obtain content identifier CID;
    Construct blockchain transaction $Tx = (h, CID, M)$;
    Submit Tx to blockchain network and obtain $TxID$;
    Update status in $\mathcal{C}$ with confirmed blockchain state;
  Return $TxID$;
else
  if *R is a query request* **then**
    Check $\mathcal{CF}$ for existence of file hash h;
    if $h \notin \mathcal{CF}$ **then**
      Return "File does not exist";
    else
      if *File F exists in cache $\mathcal{C}$* **then**
        Retrieve F and M directly from $\mathcal{C}$;
        Set response $Q_r = (F, M)$;
      else
        Query blockchain ledger to get CID and M;
        Retrieve file F from IPFS using CID;
        Update cache $\mathcal{C}$ with (h, F, M);
        Set response $Q_r = (F, M)$;
      Return Q_r;
  else
    Process other system requests according to access control policies;

3.2. Storage Process of User Credit Data

After the evaluation of a user's credit data is completed, the credit system generates a credit report file to be stored. In the initial phase, the client is required to register with the storage system and subscribe to relevant event notifications. Upon successful registration, the client is able to submit storage requests to the system at any time.

Once a storage request is received, the file metadata are immediately written to the distributed cache, and the system promptly returns an acknowledgment to the client. Subsequently, the storage system asynchronously stores the encrypted credit file in the IPFS cluster and records the corresponding key metadata on the blockchain. Finally, the client obtains the final storage result through the event-listening mechanism. The complete workflow of credit file registration, storage, and on-chain recording is illustrated in Figure 3.

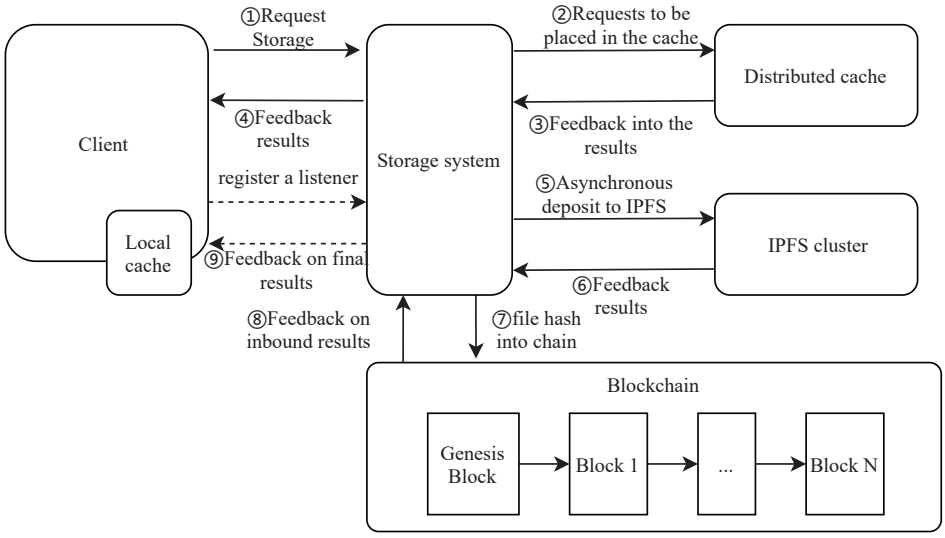


Figure 3. Credit Data Storage Process.

The detailed storage procedure of credit data is described as follows.

First, a bank submits a storage request through the client to store an encrypted credit file associated with a specific user. Upon receiving the request, the storage system writes the encrypted credit file into the cache and immediately returns an initial storage acknowledgment to the client. Subsequently, the storage system initiates an asynchronous processing thread and promptly responds to the requesting client without blocking. The asynchronous thread then uploads the encrypted credit file to the IPFS cluster. After the file is successfully stored, the IPFS cluster returns the corresponding storage result to the storage system.

Upon receiving the response from the IPFS cluster, the storage system records key metadata—such as the file index, file hash, file name, file keywords, and remarks—onto the blockchain to ensure data integrity and traceability. The blockchain system then returns the on-chain storage result to the storage system. Finally, after all storage operations are completed, the storage system notifies the requesting client of the final storage outcome through an event-driven notification mechanism.

4. Key System Design

4.1. Multi-Level Data Storage Architecture

The proposed system adopts an innovative multi-level data storage architecture designed to improve reliability, query efficiency, and overall system performance. The first storage level is a local cache, whose primary role is to support retry operations by temporarily retaining locally cached files in the event of failures during asynchronous storage processes. This mechanism ensures the reliability and eventual consistency of data storage across on-chain and off-chain components.

The second level is the distributed caching layer, which consists of a file cache and a Cuckoo filter. The file cache retains recently and frequently accessed encrypted credit files and employs a periodic eviction strategy to maintain cache availability and capacity. To mitigate storage performance bottlenecks arising from the limited throughput of the IPFS

cluster and the blockchain system, a caching mechanism combined with asynchronous storage processing is introduced, resulting in a significant improvement in overall storage efficiency. In addition, the integration of a Cuckoo filter further enhances system query performance by enabling fast existence checks with minimal storage overhead.

Specifically, the distributed caching layer stores three types of data: (1) the encrypted credit file F itself, (2) key metadata M associated with F, including owner, timestamp, and access permissions, and (3) the fingerprint of the file hash h in the Cuckoo filter $\mathcal{CF}$ for rapid existence verification. Cache consistency is ensured by an asynchronous confirmation mechanism: when a file is first written to the cache, it is also asynchronously submitted for confirmation to the IPFS cluster and blockchain. After the on-chain storage is confirmed, the cache status is updated to reflect the confirmed status of the blockchain, which ensures eventual consistency between the cache and the storage. Moreover, the Cuckoo filter $\mathcal{CF}$ works in collaboration with the cache to enable fast existence checks of a file hash, making it possible to have efficient queries and avoiding unnecessary accesses to the IPFS cluster or blockchain. This also allows light nodes to perform near full-data queries with low storage costs. The entire process of cache usage and file storage is represented in Algorithm 1.

The IPFS cluster is tasked with the responsibility of storing all the encrypted credit files uploaded by the users, while on the other hand, the blockchain system is responsible for storing the key metadata such as the hash values of the encrypted credit files. The three levels of storage provide a hierarchical and complementary storage system, as shown in Figure 4.

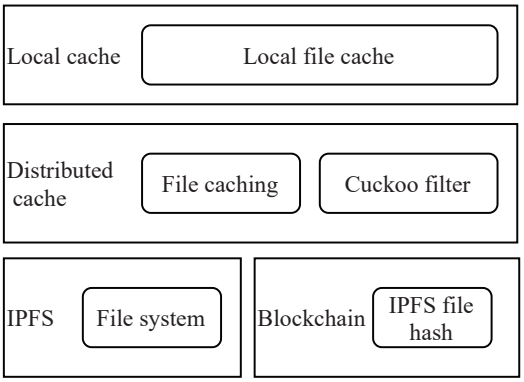


Figure 4. System multi-level storage design.

The local cache is created upon submission of a credit file storage request by the client and is cleared once the credit file has been successfully stored in the IPFS cluster and the corresponding key metadata have been recorded on the blockchain. Its primary purpose is to ensure the reliable and consistent completion of both on-chain and off-chain storage operations for credit files, particularly in the presence of asynchronous processing failures. The IPFS cluster addresses the inherent storage capacity limitations of blockchain systems by providing scalable off-chain storage. Through the dynamic addition of storage nodes, the IPFS cluster can elastically expand its storage capacity to accommodate growing data volumes. When combined with the immutability guarantees of the blockchain system, the overall availability, integrity, and security of credit files are further strengthened.

4.2. Full-Data Query Scheme for Light Nodes

In blockchain systems, full nodes maintain the complete ledger data from the genesis block onward, including transaction records, block headers, and other associated information. In contrast, light nodes store only a subset of the blockchain data, typically limited to block headers. Due to the absence of a full ledger replica, light nodes are inherently incapable of performing comprehensive historical data queries, such as retrieving arbitrary

credit files, in the same manner as full nodes. This limitation significantly restricts the practical applicability of light nodes in data-intensive business scenarios.

To overcome this challenge, this paper proposes a lightweight and efficient full-data query scheme for credit files on light nodes based on a Cuckoo filter. By maintaining compact fingerprint representations of credit files, the proposed scheme enables light nodes to rapidly verify the existence of a specified credit file within the blockchain system without requiring access to complete on-chain data. As a result, full-data query capability is logically enabled for light nodes with minimal storage overhead and negligible impact on system performance. The workflow of the proposed storage and query scheme is illustrated in Figure 5.

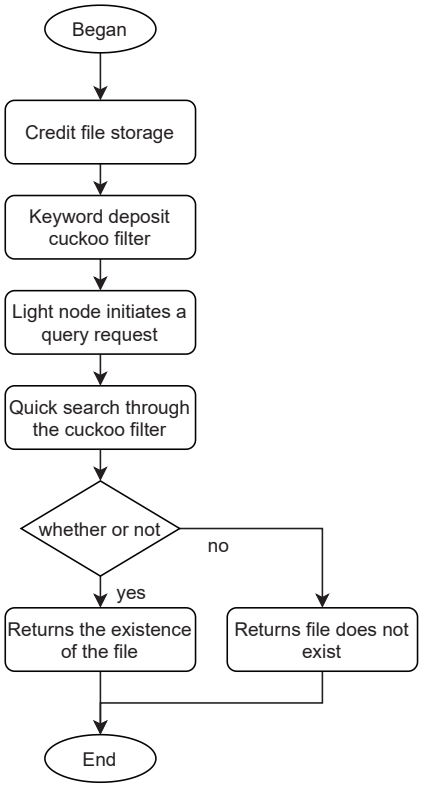


Figure 5. Light node full-volume data query process.

To facilitate efficient existence verification and full data queries for light nodes, the Cuckoo filter $\mathcal{CF}$ is used as the indexing scheme. Unlike traditional Bloom filters, the Cuckoo filter allows for the deletion of items, which is crucial for our multi-level cache eviction strategies.

Parameter Selection and Mathematical Derivation: The filter is configured with a bucket size of $b = 4$ and a fingerprint length of f bits. The theoretical upper bound of the false-positive rate ϵ is approximated by:

$$\epsilon \approx \frac{2b}{2^f} \tag{1}$$

To maintain a target $\epsilon \leq 10^{-4}$ (i.e., less than 0.01% probability of false hits), the required fingerprint length is derived as:

$$f \geq \lceil \log_2(2b/\epsilon) \rceil \approx \lceil \log_2(8/0.0001) \rceil \approx 16.3 \text{ bits} \tag{2}$$

Consequently, we selected $f = 16$ bits and $b = 4$, along with a maximum relocation threshold of $r = 500$ to balance insertion success rates under high load.

Performance Trade-offs: The choice of these parameters is done with the following key trade-offs:

- Fingerprint Length (f): Increasing f reduces the false positive rate ϵ but increases memory usage. We chose $f = 16$ to keep ϵ small while using low memory.
- Bucket Size (b) and Relocation Threshold (r): Larger buckets and a higher r enhance the load factor α and the success rate of insertions under bursty workloads, with a minor increase in computation overhead per insertion.
- Load Factor (α): We target $\alpha \approx 95\%$. Exceeding this level lowers memory efficiency and sharply increases the risk of insertion failures.

Memory Overhead Analysis: For a system scale of $N = 10^7$ stored files, the expected memory consumption is:

$$\text{Mem} \approx \frac{N \times f}{\alpha} \approx \frac{10^7 \times 16 \text{ bits}}{0.95} \approx 20 \text{ MB} \quad (3)$$

This very low memory footprint (<20 MB) enables light nodes to maintain the entire query index in memory with $O(1)$ query resolution, without needing to store the full blockchain ledger.

4.3. Security and Privacy Considerations

In this system, access control and management are primarily handled by the underlying blockchain infrastructure, so they will not be elaborated on here. All credit files in the caching layer are fully encrypted, with no plaintext data kept, laying a solid foundation for security. Even so, potential risks still require careful consideration. To further strengthen security and privacy protection, we establish a threat model and analyze possible attack vectors.

The specific cryptographic methods for file encryption and decryption, including key management and access authorization policies, are beyond the scope of this study. Nonetheless, these mechanisms form the basis for safeguarding sensitive credit data, ensuring confidentiality and privacy in practical deployment.

To strengthen the discussion of security and privacy in our system, we introduce a threat model and analyze potential attack scenarios. The primary threats to the proposed credit data storage system include:

1. Unauthorized access attacks: Efforts to gain access to unencrypted credit data without proper authorization.
2. Data tampering: Attempts to alter cached files, data stored on IPFS, or blockchain metadata.
3. Replay and double-spending: Maliciously resubmitting past transactions to undermine system consistency.
4. Denial-of-service (DoS): Sending high-volume requests to the cache, IPFS, or blockchain, which degrades system performance.

Corresponding mitigation strategies are listed below:

1. Encrypt all credit files before caching or uploading them to IPFS, thus preventing unauthorized access.
2. Store immutable file metadata (including hashes, ownership, and permissions) on the blockchain to detect any tampering behavior.
3. Access control and permission verification are implemented through the underlying permissioned consortium model of Hyperledger Fabric, which includes endorsement policies customized for each participating organization.
4. The caching layer, along with the asynchronous task queue, lessens the impact of sudden spikes in malicious requests. What's more, the Cuckoo filter cuts down on

unnecessary access to backend systems, in turn boosting the system's resilience against DoS attacks.

This threat analysis indicates that the proposed system is designed to maintain the confidentiality, integrity, and availability of credit data—this confirms its suitability for use in sensitive financial environments.

5. System Testing and Analysis

5.1. Experimental Setup

The proposed system is implemented using Hyperledger Fabric v2.5.11 as the underlying blockchain framework. Fabric is adopted in this study under a permissioned consortium blockchain model, which is consistent with the requirements of financial and credit data sharing scenarios. In this setting, all participating nodes belong to known and authenticated organizations, such as banks, credit agencies, and regulatory institutions. Access control and identity management are enforced through Fabric's Membership Service Provider (MSP), and transaction validity is guaranteed by organization-specific endorsement policies. This consortium-based model ensures controlled participation, accountability, and compliance with regulatory requirements, while avoiding the performance overhead and privacy risks associated with public permissionless blockchains. These assumptions form the underlying trust and security foundation of the proposed system.

Following this architectural design, the blockchain network cluster is constructed using Docker (v20.10.7), and the corresponding chaincode is developed in Java (JDK 11). An IPFS cluster is deployed based on Go-IPFS version 0.28.0, while MongoDB v7.0 is adopted as the distributed caching system. The server-side storage service is implemented using Spring Boot (v2.7.18), and the Fabric Gateway (v1.7.0) is employed to invoke and manage chaincode interactions within the Fabric network.

The experimental environment is deployed on a laptop equipped with an 11th Gen Intel® Core™ i7-1165G7 processor (2.80 GHz), 16 GB RAM, and a 500 GB SSD. To simulate a realistic deployment environment, VMware Workstation Pro is used to configure a virtualized CentOS system with 8 CPU cores, 8 GB RAM, and 20 GB of disk space. Within this virtual environment, Docker is installed to deploy the blockchain network, caching system, IPFS cluster, and other system components required for performance evaluation and functional testing. Although the experiments are conducted on a single-node virtualized environment, the relative performance trends remain indicative, as both schemes are evaluated under identical hardware and network conditions.

To rigorously evaluate the performance improvements achieved by the proposed system, we compare it against a representative baseline scheme. This baseline adopts the standard synchronous integration architecture that is widely used in existing studies [22–25] and industrial applications. The Baseline Scheme follows a typical workflow: file storage and blockchain metadata registration are performed synchronously, with no intermediate caching or probabilistic indexing mechanisms in place. Specifically, the client must wait for the IPFS upload to complete and the blockchain transaction to be committed before a request is finalized. We selected this standard implementation as the comparative baseline because it represents the fundamental performance boundary of Blockchain–IPFS systems. Comparing against this canonical model allows us to quantify the specific contributions of our multi-level caching and Cuckoo filter mechanisms. The specific architectural differences are summarized in Table 1.

Due to hardware and deployment constraints, the current experiments are conducted in a single-node virtualized environment. While this setup does not fully capture the network latency and consensus overhead inherent in large-scale distributed deployments, it provides a controlled proof-of-concept evaluation. In a real-world multi-node environment, additional latency introduced by Raft consensus coordination and IPFS P2P data propagation would inevitably increase the absolute response time of both the Baseline and the proposed scheme. However, because the proposed framework decouples transaction submission from storage confirmation through asynchronous processing and multi-level caching, part of the backend communication latency can be effectively masked from the client side. In contrast, the synchronous Baseline approach directly exposes consensus and storage delays to users. Therefore, although the absolute latency is expected to increase in distributed settings, the architectural advantages of the proposed scheme are likely to remain stable and even more pronounced under multi-node deployments. Future work will extend this evaluation to verify these scalability considerations.

Table 1. Architectural comparison between the proposed scheme and the baseline IPFS–blockchain scheme.

Aspect	Proposed Scheme	Baseline IPFS–Blockchain Scheme
Storage mode	Multi-level caching with asynchronous persistence	Synchronous IPFS and blockchain storage
Light-node query capability	Supported via Cuckoo filter	Not supported
Query path	Cache- and filter-based lookup	Direct blockchain and IPFS access
Burst traffic handling	Asynchronous queue with caching	Not supported
On-chain data	File metadata only	File metadata only
Design focus	Query efficiency and system robustness	Functional data storage

5.2. System Performance Evaluation

Based on the architectural differences summarized in Table 1, the following experiments quantitatively evaluate the performance advantages of the proposed scheme.

To comprehensively assess the performance of the proposed system, experimental evaluations are conducted from three key perspectives: data storage, data retrieval, and data download. A comparative analysis is performed between the proposed prototype system and a conventional blockchain–IPFS integrated solution, which is hereafter referred to as the baseline scheme.

Considering that the size of typical credit files in real-world financial applications generally ranges from 1 MB to 50 MB, a broader range of data scales is selected to evaluate system performance under diverse workload conditions. Specifically, the file sizes used in the experiments are configured as 1 KB, 1 MB, 10 MB, 50 MB, 100 MB, and 500 MB, thereby covering both lightweight metadata-like files and large-scale credit data files.

The experimental evaluation encompasses the complete lifecycle of credit file operations, including file storage, indexing and retrieval, and data download. By measuring and comparing the execution time and response performance at each stage, the overall efficiency and scalability of the proposed scheme are systematically analyzed. The comparative performance results of the proposed scheme and the baseline scheme are summarized in Table 2.

Based on the performance results reported in Table 2, several key observations can be made regarding storage, query, and download performance.

For the baseline scheme, storage latency consistently exceeds 2 s, as file storage in the IPFS cluster and metadata registration on the blockchain are executed synchronously. In contrast, the proposed scheme adopts a cache-first strategy combined with non-blocking

asynchronous processing. Specifically, files are initially written to the cache, while persistence to IPFS and metadata recording on the blockchain are completed asynchronously. As a result, for files smaller than 100 MB, the storage latency of the proposed scheme ranges from 7 ms to 1667 ms, exhibiting a gradual and predictable increase with file size and demonstrating a significant performance improvement over the baseline scheme. In the experiments, credit file names are used as the primary query keys. When the number of records is below 1000, the baseline scheme requires approximately 21–28 ms to complete a query. By incorporating a Cuckoo filter, the proposed scheme reduces the query latency to approximately 1 ms, regardless of file size. This substantial improvement is mainly attributed to the efficient membership verification capability of the Cuckoo filter, which avoids frequent access to the blockchain and IPFS systems.

Table 2. Performance comparison between the proposed scheme and the baseline IPFS–blockchain scheme.

File Size	Programme	Storage/ms	Query/ms	Download/ms
1 K	Proposed Scheme	7	1	5
1 K	Baseline IPFS–blockchain scheme	2046	21	3
1 M	Proposed Scheme	68	1	15
1 M	Baseline IPFS–blockchain scheme	2086	22	15
10 M	Proposed Scheme	322	1	82
10 M	Baseline IPFS–blockchain scheme	2089	23	119
50 M	Proposed Scheme	941	1	458
50 M	Baseline IPFS–blockchain scheme	2337	26	577
100 M	Proposed Scheme	1667	1	810
100 M	Baseline IPFS–blockchain scheme	2543	28	1148
500 M	Proposed Scheme	9102	1	3776
500 M	Baseline IPFS–blockchain scheme	4405	29	5988

For file download operations, the baseline scheme retrieves data directly from the IPFS cluster, resulting in download latencies ranging from 3 ms to 1148 ms. In contrast, the proposed scheme achieves download latencies between 5 ms and 810 ms, benefiting from the caching layer that intercepts repeated access requests for recently and frequently accessed files. Consequently, the proposed scheme exhibits improved download performance, particularly for small- and medium-sized files.

In the experiment involving a file size of 500 MB, the storage latency of the proposed scheme increases to 9102 ms, compared with 4405 ms for the baseline scheme. This degradation can be attributed to the additional overhead introduced by cache management and asynchronous task coordination when handling large-scale files. Although the proposed scheme still outperforms the baseline scheme in terms of query and download performance, its storage advantage diminishes in this scenario. Nevertheless, considering that typical credit files in financial systems are generally smaller than 100 MB, the proposed scheme remains well suited for practical credit data management scenarios that involve high-frequency access to small- and medium-sized files.

Among all system functionalities, query operations are the most frequently invoked. To comprehensively evaluate the query performance under realistic workload conditions, additional experiments were conducted to measure query latency under varying levels of concurrency using the prototype system deployed on the specified hardware platform. The tested concurrency levels were set to 10, 50, 100, 200, 500, 1000, 1500, and 2000 concurrent requests, and the corresponding results are presented in Figure 6.

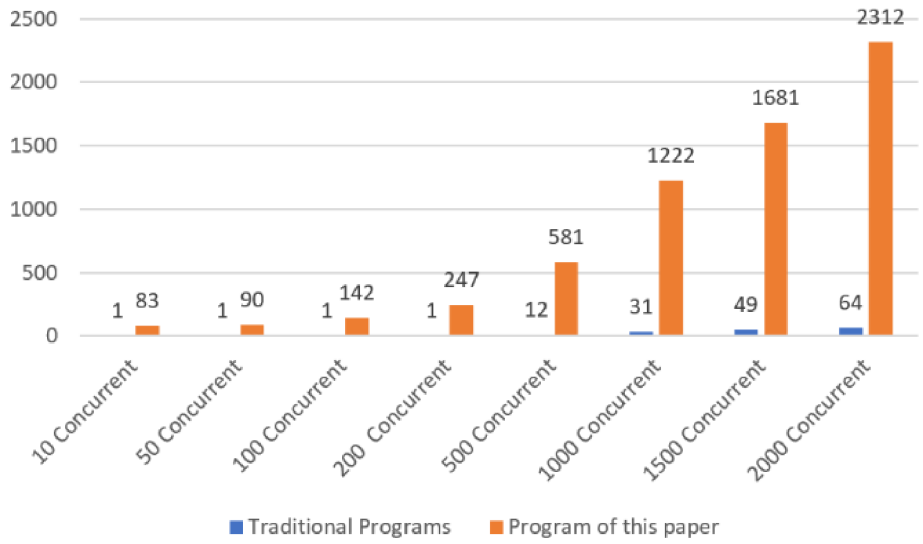


Figure 6. Performance test results

5.3. Query Path Complexity and Asynchronous Processing Overhead Analysis

In the storage system proposed in this study, a Cuckoo filter is introduced to determine the existence of credit files, thereby avoiding frequent query requests to the IPFS system and the blockchain network. From an algorithmic complexity perspective, the Cuckoo filter determines whether a credit file exists by computing two candidate buckets using hash functions. Consequently, the query operation exhibits O(1) time complexity, which is independent of the total number of files stored in the system. In practical implementation, each query mainly consists of three steps: (1) computing the fingerprint of the query keyword using a hash function; (2) calculating the positions of two candidate buckets via two independent hash functions; and (3) checking whether the fingerprint exists in either of the candidate buckets. This mechanism does not involve full-table scanning or traversal. As a result, the system maintains stable and predictable query performance even as the system scale increases. This property makes the proposed approach particularly suitable for blockchain light-node scenarios, where storing a complete ledger locally is impractical.

To further enhance system scalability and robustness under burst traffic conditions, a Java-based asynchronous processing mechanism is adopted to decouple user storage requests through an asynchronous queue. This design effectively mitigates the risk of system congestion or failure caused by sudden traffic surges, albeit at the cost of introducing additional system overhead. From a resource consumption perspective, the overhead introduced by the asynchronous mechanism mainly originates from the following aspects.

Thread resource overhead: Asynchronous tasks are executed using a dedicated thread pool. According to common JVM implementations, each worker thread is allocated an independent thread stack with a default size of approximately 1 MB. Since the thread pool is configured with an upper bound on the number of threads, the total thread-related overhead remains controllable and predictable.

Asynchronous queue overhead: The asynchronous queue temporarily stores pending storage requests, along with key metadata and contextual state information associated with the files to be stored. Experimental observations indicate that each additional queue element consumes approximately several hundred bytes of memory, which is relatively modest and scales linearly with the queue length.

Context switching and scheduling overhead: The adoption of asynchronous processing inevitably introduces thread context switching and scheduling costs. However,

by leveraging a thread pool mechanism, excessive thread creation is avoided, thereby effectively mitigating potential performance degradation.

In summary, although the introduction of asynchronous processing incurs a certain amount of system overhead, its non-blocking characteristics substantially enhance the system's ability to handle high-concurrency and burst traffic scenarios. When combined with the proposed Cuckoo filter and caching mechanisms, the overall system design achieves significant improvements in both query efficiency and storage performance, while maintaining acceptable resource consumption.

5.4. Discussion and Critical Reflection

The experimental results demonstrate that the proposed storage system achieves noticeable performance and architectural advantages over conventional approaches. However, the proposed design also exhibits certain limitations. This section provides a critical discussion of the underlying design assumptions, applicable scenarios, and potential constraints of the proposed method, rather than questioning its overall effectiveness.

1. Consistency vs. Availability Trade-off: The system uses an asynchronous task queue to isolate file uploads from blockchain transaction processing. This choice prioritizes availability, stability, and response efficiency rather than immediate strong consistency. As a result, a file may temporarily exist in the local cache before it is formally recorded on the blockchain, leading to a short period during which the cache and blockchain are not fully synchronized. Nevertheless, the file will eventually be written to the blockchain through the asynchronous background mechanism. For scenarios requiring strict consistency, such as real-time monitoring or auditing, this can be seen as a limitation. However, in most credit data-sharing applications, this kind of temporary inconsistency only lasts a few seconds and has almost no practical impact on system performance.
2. Scalability of the In-Memory Index: To achieve constant-time query performance with $O(1)$ complexity, the Cuckoo filter is kept in memory. As noted in Section 4.2, indexing roughly 10^7 credit files takes about 20 MB of memory—this is acceptable for blockchain light nodes. However, if the deployment scale far exceeds the workload assumed in this study, the filter's memory usage will grow linearly with the number of indexed files. For extremely large-scale storage scenarios, future research can explore additional optimizations: these include filter sharding, hierarchical partitioning, or hybrid indexing approaches that combine in-memory structures with persistent storage, all intended to improve scalability.
3. False-Positive Characteristics of the Cuckoo Filter: As a probabilistic data structure, the Cuckoo filter exhibits a controllable but non-zero false-positive rate ($\epsilon \approx 10^{-4}$ in our configuration). Under the configuration adopted in this study, there exists a very small probability that a query is classified by the filter as "present" while no corresponding record exists in the blockchain or backend storage. It should be emphasized that such false positives only lead to a small number of redundant lookup operations and do not compromise data integrity, correctness, or security. Compared with standard Bloom filters, which also suffer from false positives but do not support safe deletion, the Cuckoo filter provides dynamic deletion capability, higher query efficiency, and better space utilization. These characteristics make it more suitable for the dynamic storage and querying of credit files considered in this work.

6. Conclusions

This paper proposes a blockchain–IPFS-based framework for storing and sharing banking credit data, which incorporates multi-level caching, asynchronous processing,

and a Cuckoo filter to tackle key limitations of traditional blockchain storage systems. It focuses on three main challenges: light nodes cannot perform full-data queries, storage and query performance is limited, and robustness is weak under burst traffic. First, a full-data query mechanism based on the Cuckoo filter lets light nodes efficiently verify file existence and access associated metadata without keeping full ledger copies. Second, a multi-level storage architecture with non-blocking asynchronous processing separates user requests from on-chain and off-chain operations, greatly improving throughput while reducing storage latency. Third, caching strategies and asynchronous task queues enhance system resilience in high-concurrency and burst traffic scenarios.

Experimental results show that, for files smaller than 100 MB, the proposed scheme outperforms traditional blockchain–IPFS-based solutions significantly in terms of storage latency, query efficiency, and system stability. The results also offer an important engineering insight: while caching mechanisms effectively boost performance for small and medium-sized files, they may introduce storage overhead in large-file scenarios.

Future work will focus on strengthening security and privacy protection mechanisms—including fine-grained access control and encryption strategies—to further improve data confidentiality and applicability in real-world financial systems.

Author Contributions: Conceptualization, J.Y. and M.Z.; methodology, J.Y.; software, J.Y.; validation, J.Y., H.L. and M.Z.; formal analysis, J.Y.; investigation, J.Y.; data curation, J.Y.; writing—original draft preparation, J.Y.; writing—review and editing, J.Y., H.L. and M.Z.; visualization, J.Y.; supervision, H.L.; project administration, H.L. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The raw data supporting the conclusions of this article will be made available by the authors on request.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Yang, H. *Better Utilize the Power of Digital Finance and Work Together to Make "Five Big Articles"*; China Banking: Beijing, China, 2024; pp. 35–37.
2. Zhao, J.; Wu, Q. The new wave of digitisation and ideas for commercial banks to cope with it. *Int. Financ.* **2022**, 6, 69–73. [CrossRef]
3. Liang, W. Financial Credit Data Sharing: Realistic Dilemma and Future Scenario. *Credit. Ref.* **2019**, *37*, 14–19.
4. He, L. *Breaking Down "Barriers" to Data Sharing Moves Forward*; China Credit: Beijing, China, 2019; pp. 108–109.
5. Dong, C. Research on Social Credit Information Sharing Based on Blockchain and Federated Learning. Master's Thesis, Yunnan University of Finance and Economics, Kunming, China, 2023.
6. Lu, S. Social credit data management and application in the new development phase: A perspective based on blockchain and privacy computing technologies. *Credit. Ref.* **2023**, *41*, 61–65.
7. Ju, C.; Zou, J.; Fu, X. Research on the Design and Application of Big Data Credit Platform Incorporating Blockchain Technology. *Comput. Sci.* **2018**, *45*, 522–526+552.
8. He, P.; Yu, G.; Zhang, Y.; Bao, Y. An overview of blockchain technology and application foresight. *Comput. Sci.* **2017**, *44*, 1–7+15.
9. Yuan, Y.; Wang, F. Blockchain Technology Development Status and Prospects. *Acta Autom. Sin.* **2016**, *42*, 481–494. [CrossRef]
10. Shao, Q.; Jin, C.; Zhang, Z.; Qian, W.; Zhou, A. Blockchain Technology: Architecture and Advances. *Chin. J. Comput.* **2018**, *41*, 969–988.
11. Nakamoto, S. *Bitcoin: A Peer-to-Peer Electronic Cash System*; Consulted: Anstruther, UK, 2008.
12. Yang, J.; Hou, H.; Li, H.; Zhu, Q. *One Method for Implementing Privacy Protection of Electronic Invoices Based on Blockchain*; IEEE: New York, NY, USA, 2021; pp. 99–104.

13. Meng, W.T.; David, Z. Hyperledger Fabric Consensus Mechanism Optimisation Scheme. *Acta Autom. Sin.* **2021**, *47*, 1885–1898. [CrossRef]
14. Manevich, Y.; Barger, A.; Tock, Y. Endorsement in Hyperledger Fabric via service discovery. *IBM J. Res. Dev.* **2019**, *63*, 2:1–2:9. [CrossRef]
15. Trautwein, D.; Raman, A.; Tyson, G.; Castro, I.; Scott, W.; Schubotz, M.; Gipp, B.; Psaras, Y. Design and evaluation of IPFS: A storage layer for the decentralized web. In *Proceedings of the ACM SIGCOMM 2022 Conference, New York, NY, USA, 22–26 August 2022*; ACM: New York, NY, USA, 2022; pp. 739–752. [CrossRef]
16. Bloom, B.H. Space/time trade-offs in hash coding with allowable errors. *Commun. ACM* **1970**, *13*, 422–426. [CrossRef]
17. Fan, B.; Andersen, D.G.; Kaminsky, M.; Mitzenmacher, M.D. Cuckoo Filter: Practically Better Than Bloom. In *Proceedings of the 10th ACM International on Conference on Emerging Networking Experiments and Technologies* (CoNEXT '14), Sydney, Australia, 2–5 December 2014; ACM: New York, NY, USA, 2014; pp. 75–88. [CrossRef]
18. Reviriego, P.; Martinez, J.; Larrabeiti, D.; Pontarelli, S. Cuckoo Filters and Bloom Filters: Comparison and Application to Packet Classification. *IEEE Trans. Netw. Serv. Manag.* **2020**, *17*, 2690–2701. [CrossRef]
19. Wang, H.; Dai, H.; Chen, S.; Chen, Z.; Chen, G. A Review of Research on Filter Data Structures. *Comput. Sci.* **2024**, *51*, 35–40.
20. Zheng, Q.; Li, Y.; Chen, P.; Dong, X. An Innovative IPFS-Based Storage Model for Blockchain. In *Proceedings of the 2018 IEEE/WIC/ACM International Conference on Web Intelligence (WI), Santiago, Chile, 3–6 December 2018*; IEEE: New York, NY, USA, 2018; pp. 704–708. [CrossRef]
21. Khalid, M.I.; Ehsan, I.; Al-Ani, A.K.; Iqbal, J.; Hussain, S.; Ullah, S.S.; Nayab. A Comprehensive Survey on Blockchain-Based Decentralized Storage Networks. *IEEE Access* **2023**, *11*, 10995–11015. [CrossRef]
22. Nizamuddin, N.; Hasan, H.R.; Salah, K. IPFS-Blockchain-Based Authenticity of Online Publications. In *Proceedings of the Blockchai—ICBC 2018, Seattle, WA, USA, 25–30 June 2018*; Chen, S., Wang, H., Zhang, L.J., Eds.; Springer: Cham, Switzerland, 2018; pp. 199–212.
23. Chen, L.; Zhang, Y.; Zou, G.; Sha, D.; Lu, Z. Blockchain-based Multi-level Credit Data Secure Storage and Sharing System. *Softw. Guide* **2024**, *23*, 150–156. Available online: https://kns.cnki.net/kcms2/article/abstract?v=l4rz0Y_R2D8QNEo-KsDGRYOXN6C3w_XzDBhfIJNmej8L-DSdSnwkPODiiclVep3EFV254FJzSOO6HB5dvdrxXXAaTW_cXzi3ORZrDK3PQXhkA4JUBg-7865nbTSlA_2c87imF0NLz6AYRmUX-llp-ZlLc0aYiDJRBS67ZQS63ZQ=&uniplatform=NZKPT (accessed on 5 March 2026). [PubMed]
24. Fan, X.; Fan, C.; Wu, Y. Enabling Food Supply Chain Privacy Information Protection Based on Blockchain and IPFS Technologies. *J. Appl. Sci.* **2019**, *37*, 179–190.
25. Rui, K.; Pan, H.; Shu, S. Design of data storage system based on blockchain and IPFS protocols. *J. Chang. Inst. Technol. Nat. Sci. Ed.* **2023**, *24*, 121–124.
26. Heo, J.W.; Ramachandran, G.S.; Dorri, A.; Jurdak, R. Blockchain Storage Optimisation with Multi-Level Distributed Caching. *IEEE Trans. Netw. Serv. Manag.* **2022**, *19*, 3724–3736. [CrossRef]
27. Li, C.; Zhang, J.; Yang, X. Scalable blockchain storage mechanism based on two-layer structure and improved distributed consensus. *J. Supercomput.* **2022**, *78*, 4850–4881. [CrossRef]
28. Li, R.; Song, T.; Mei, B.; Li, H.; Cheng, X.; Sun, L. Blockchain for Large-Scale Internet of Things Data Storage and Protection. *IEEE Trans. Serv. Comput.* **2019**, *12*, 762–771. [CrossRef]
29. Dai, M.; Zhang, S.; Wang, H.; Jin, S. A Low Storage Room Requirement Framework for Distributed Ledger in Blockchain. *IEEE Access* **2018**, *6*, 22970–22975. [CrossRef]

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.